

Real-world (config + logging together) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Work through in order — don't jump ahead until you've genuinely tried. Each hint now has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

Where this exercise is saved: its own folder, `practice/config_logging_wiring/`, because it needs more than one file:

```
practice/config_logging_wiring/  
├─ exceptions.py      MissingConfigError  
├─ config.py          load_config() -> Config  
├─ logging_setup.py   get_logger(name)  
├─ main.py            wires the two together  
└─ .env              you create this; never committed
```

Run it: `cd practice/config_logging_wiring && python main.py` — from inside the folder, so `from config import load_config` finds the file sitting next to it.

Builds on: `practice/logging_practice.py` ([Intermediate part 2](#)) — **copy** its `get_logger()` into `logging_setup.py`, same name, same signature. And `practice/custom_errors_practice.py` ([Basic](#)) — the same custom-error pattern, re-written here as `MissingConfigError` in `exceptions.py`. Copies, not imports: this folder runs on its own. The hints below are about `main.py`, the one genuinely new file; the [Solution](#) shows all four in full.

- [Hint 1](#)
- [Hint 2](#)
- [Hint 3](#)
- [Hint 4](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 1 — What's the idea here?

Basic Version

This exercise is not about writing new code — you already built a config loader and a logger. Now `practice/config_logging_wiring/main.py` just needs to use both together: load the config, get a logger, then print a log message that includes something from the config.

Think about which one you should call first.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

This exercise is about **wiring**, not new logic — you already built `load_config()` and `get_logger()` separately. Now `practice/config_logging_wiring/main.py` just needs to call both, and use them together: get the config, get a logger, then log something that includes a value from config.

Think about the order: which one do you call first, and why does that order matter?

Difference between Basic and Intermediate: Basic says "load config, then get a logger."

Intermediate explains *why* that order matters (fail fast, before anything else runs) — this is the depth the exercise's Solution is written at.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 2 — The exact pieces

Basic Version

At the top of `practice/config_logging_wiring/main.py`, bring in both functions, each from the file sitting next to it in that folder:

- `load_config()` from `practice/config_logging_wiring/config.py`
- `get_logger()` from `practice/config_logging_wiring/logging_setup.py` — the copy of `practice/logging_practice.py`'s function

Call `load_config()` first, before anything else — if a setting is missing, you want to find out right away.

Then use an f-string to put a config value inside your log message.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

`from config import load_config` and `from logging_setup import get_logger` at the top of `practice/config_logging_wiring/main.py`. Call `load_config()` first — if it's going to fail (a missing key), you want that to happen before you've done anything else, including setting up logging.

Use an f-string to build your log message with a value from the config object: `logger.info(f"...{config.some_field}...")`.

To handle a startup failure cleanly, wrap `load_config()` in `try / except MissingConfigError as e:`, and on failure `print(f"Startup failed: {e}")` followed by `sys.exit(1)` — a normal Python program exits with code `1` to signal "something went wrong" to whatever ran it.

Difference between Basic and Intermediate: Basic just names which function to call first and why. Intermediate adds the actual failure-handling code around that call — this is the depth the exercise's Solution is written at.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 3 — The plan, in pseudocode

Basic Version

```
in practice/config_logging_wiring/main.py:
    bring in load_config and get_logger

    config = load_config()      # do this first
    logger = get_logger("main")

    log a message that includes a value from config, like the log level

run the file and check the message shows up with the config value in it
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

```
in practice/config_logging_wiring/main.py:
    import load_config from config.py
    import get_logger from logging_setup.py
    import sys and MissingConfigError

    try:
        config = load_config()          # do this first – fail fast if something'
    except MissingConfigError as e:
        print an error message and sys.exit(1)

    logger = get_logger(__name__)

    logger.info a message that includes config.log_level (or any field you have)
```

run it, and confirm the line appears on screen with the config value inside it
run it again with a broken/missing config value, and confirm you get a clean
one-line error instead of a Python traceback

Difference between Basic and Intermediate: Basic's pseudocode never fails on purpose — it assumes config always loads. Intermediate adds the failure path explicitly, with a `try / except` around the one call that can fail.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 4 — Almost the whole thing

Basic Version

```
# practice/config_logging_wiring/main.py
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger("main")

logger.info("Loaded config with log level: " + config.log_level)
```

Run it with `cd practice/config_logging_wiring && python main.py` and check the line prints.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

```
# practice/config_logging_wiring/main.py
import sys

from config import load_config
from exceptions import MissingConfigError
from logging_setup import get_logger

try:
    config = load_config()
except MissingConfigError as e:
    print(f"Startup failed: {e}")
    sys.exit(1)

logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Run it with `cd practice/config_logging_wiring && python main.py` and confirm the line prints. Then break your config on purpose (delete a required value) and confirm you get the clean one-line "Startup failed: ..." message instead of a traceback, then compare against the [Solution](#).

Difference between Basic and Intermediate: same 2 function calls (`load_config()` , `get_logger()`) at 2 completeness levels — Basic proves the wiring works, Intermediate adds a clean failure path so a broken config doesn't crash with a traceback.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Full solution: [Show me the solution](#)